

Embedded Lab

An online teaching laboratory for Microcontrollers and Embedded Systems

► I2C Interface

► I2C LCD Display

► Lab Data

► I2C I O Expander

Share

LAB 14: INTER-INTEGRATED CIRCUIT (I2C) COMMUNICATION

Posted on [May 9, 2011](#) | by [R-B](#) | [30 comments](#) | Like 60

AdChoices

► [Embedded System Tutorial](#)

► [Embedded Programming](#)

► [Lab Testing Companies](#)

AdChoices

► [Lab Data](#)

► [I2C I O Expander](#)

► [Embedded Projects](#)

AdChoices

I2C (Inter-Integrated Circuit) is a short distance serial interface that requires only 2 bus lines for data transfer. It was invented by Philips in 1980's, originally to provide easy on-board communications between a CPU and various peripheral chips in a TV set. Today, it is widely used in varieties of embedded systems to connect low speed peripherals (external EEPROMs, digital sensors, LCD drivers, etc) to the main controller. In this experiment, we will cover an overview of I2C protocol, its implementation in PIC microcontrollers, and the method of connecting single and multiple devices on a common I2C bus. We will demonstrate the technique by connecting two I2C EEPROM chips (24LC512) and an I2C compatible temperature sensor (DS1631) with PIC18F2550 microcontroller.

Search...

Ad

g+1

1

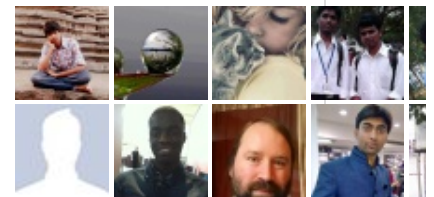
Fol



Embedded Lab

Like

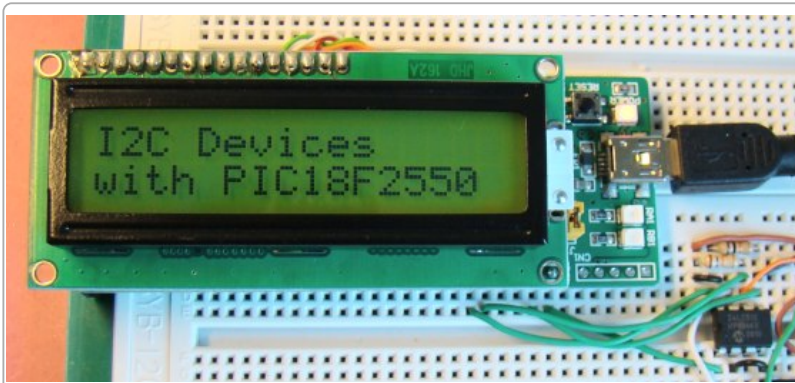
5,754 people like [Embedded Lab](#).



Facebook social plugin

[Read Our Privacy Policy!](#)

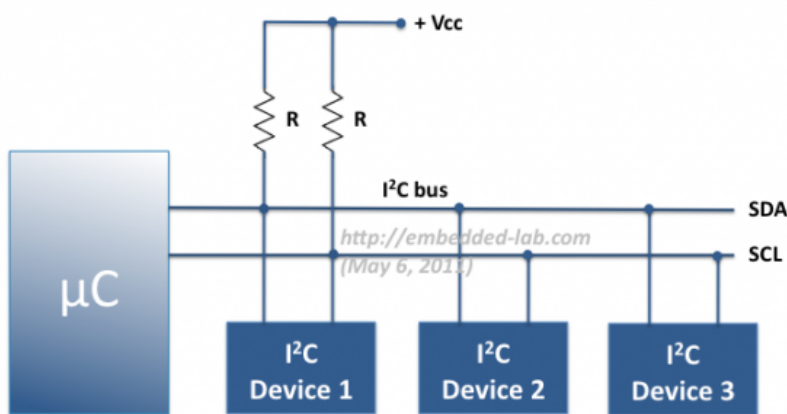
SUBSCRIBE



I2C devices with PICMicro

Theory

I2C bus has two lines: a **serial data** line (SDA) and a **serial clock** line (SCL). Any data sent from one device to another goes through the SDA line, whereas the SCL line provides the necessary synchronization clock for the data transfer. The devices on an I2C bus are either **Masters** or **Slaves**. Only a Master can initiate a data transfer and Slaves respond to the Master. It is possible to have multiple Masters on a common bus, but only one could be active at a time. The SCL clock line is always driven by the master. In this tutorial, we will discuss a single master case, and our master is the PIC18F2550 microcontroller. The figure below shows a I2C bus with a single master and three slaves. Slaves can never initiate a data transfer but they can transfer data over the I2C bus, and that is always controlled by the Master.



Multiple devices on common I2C bus

Both SCL and SDA lines are open drain drivers, and are therefore, connected to a positive supply voltage through pull-up resistors. This means the *I2C devices can only pull the line low, but they cannot drive it high*. When no device is pulling on the line, it will float high through the pull-up resistor. This is why pull-up resistors are important in I2C. The open-drain outputs of I2C devices helps to perform the wired-AND function on the bus. Data on the I2C bus

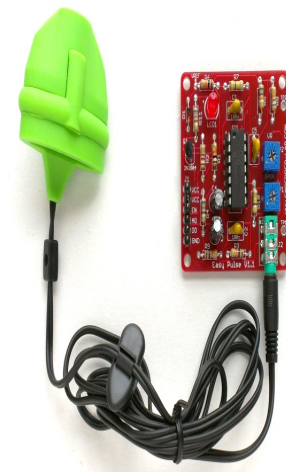


Subscribe through email

Sign Up

EASY PULSE SENSOR

Easy Pulse Sensor is designed for hobby and educational applications to illustrate the principle of finger photoplethysmography (PPG) as a non-invasive technique for detecting cardio-vascular pulse wave. [Read More ...](#)



[US customers buy here](#)

[International customers buy here](#)

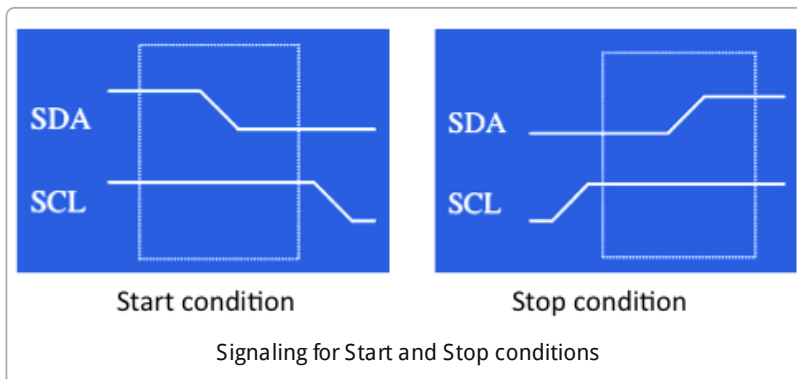
CATEGORIES

- [555 Timer](#) (8)
- [Analog](#) (2)

can be transferred at a rate up to 100 Kbps (in standard mode), 400 Kbps (in fast mode), or up to 3.4 Mbps (in high-speed mode).

Start and Stop conditions

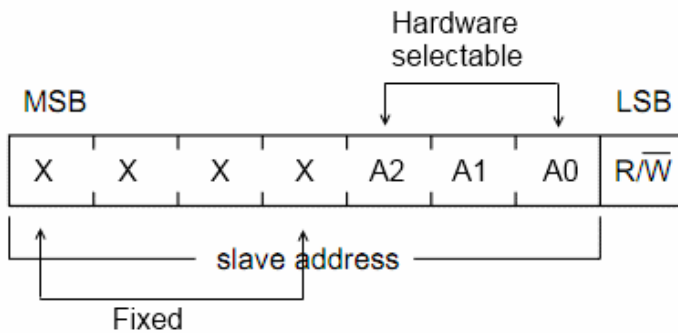
Prior to any transaction on the bus, a Start condition is issued by the Master device to inform all the slave devices that something is about to be transmitted on the bus. As a result, all connected slave devices will listen to the serial data line for instructions. The Start condition is issued by pulling the SDA line low followed by the SCL line. Once the data transfer is finished, the bus Master sends a Stop condition to inform other devices that it would like to release the bus. The signaling used for a Stop condition is a release of the SCL line followed by a release of the SDA line. Remember that when the lines are released, they float high because of the pull-up resistors. Thus, the Start and Stop sequences mark the beginning and end of a transaction with the slave device.



I2C device addressing

Each device connected to the bus is software addressable by a unique 7-bit or 10-bit address. The use of 10-bit address is not very common and therefore, is not discussed here. The first byte sent after the Start condition is known as **Control byte**. The first seven bits of the control byte make up the slave address, whereas the eighth bit (LSB) is a data direction bit (R/W): a 'zero' in the LSB of the first byte indicates that the Master will write information to a selected slave. A 'one' in this position indicates that the Master will read data from the slave. For 7-bit devices, typically the first four bits are fixed, the next three bits are set by hardware address pins (A0, A1, and A2) that allow the user to modify the I2C address allowing up to eight of the same devices to operate on the I2C bus. These pins are held high to VCC or held low to GND. We will discuss more in the circuit section on how to setup the device address on the bus.

- [Arduino](#) (41)
- [AVR Projects](#) (35)
- [AVR Tutorials](#) (5)
- [chipKIT](#) (13)
- [dsPIC](#) (1)
- [Embedded Lab Projects](#) (58)
- [Embedded Labs](#) (25)
- [Embedded Lessons](#) (39)
- [MCU development tools](#) (8)
- [Microcontroller Programmers](#) (6)
- [MSP430 Launchpad](#) (2)
- [Netduino](#) (8)
- [PIC Projects](#) (76)
- [PIC Tutorials](#) (43)
- [PIC18F](#) (10)
- [Power Supply](#) (6)
- [Processing](#) (3)
- [Product Review](#) (14)
- [Products](#) (29)
- [Raspberry Pi](#) (2)
- [Robotics](#) (4)
- [Tech News](#) (86)
- [Tips and Tricks](#) (62)
- [Uncategorized](#) (1)

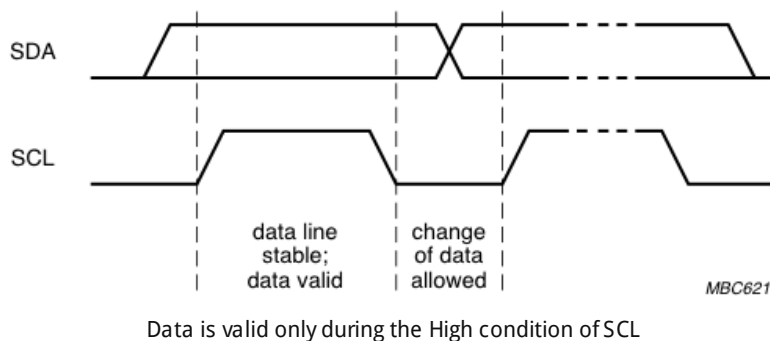


The control byte (first byte after the Start condition) holds the slave address

When a control byte is sent, each device in the system compares the first seven receiving bits with its address. If they match, the device considers itself addressed by the master as a slave-receiver or slave-transmitter, depending upon the value of the data direction bit (8th bit).

Data transfer

Every byte put on the SDA line must be 8-bits long. The data is sent on the SDA line starting with the most significant bit (MSB) first and the SCL line produces a synchronization clock. The data on the SDA line is considered valid when SCL is high, and therefore the data must be stable during the HIGH period of the clock. The HIGH or LOW state of the data line can only change when the clock signal on the SCL line is LOW. This is how the timing of each bit works.



If a slave is not in a position to receive or transmit another complete byte of data until it has performed some other function, for example servicing an internal interrupt, it can hold the SCL line low to force the master into a wait state. Data transfer continues when the slave is ready for another byte of data and releases the clock line.

A data transfer is always terminated by a Stop condition generated by the master. However, if a master still wishes to communicate

on the bus, it can generate a *repeated-Start condition* and address another slave without first generating a Stop condition.

Acknowledgment

Usually, a receiver which has been addressed is obliged to generate an acknowledge (ACK) after each byte has been received. The acknowledge takes place after the 8th data bit has been transferred in any transaction. During this state the transmitter should release the SDA bus to allow the receiver to drive it. The receiver drives the SDA signal low to acknowledge receipt of the byte. If the receiver does not drive SDA low, the condition is a no-acknowledge (NACK) and the operation is aborted. If the byte sent is the control byte (slave address + R/W bit), then only that slave which has the matching address will respond with an acknowledge.

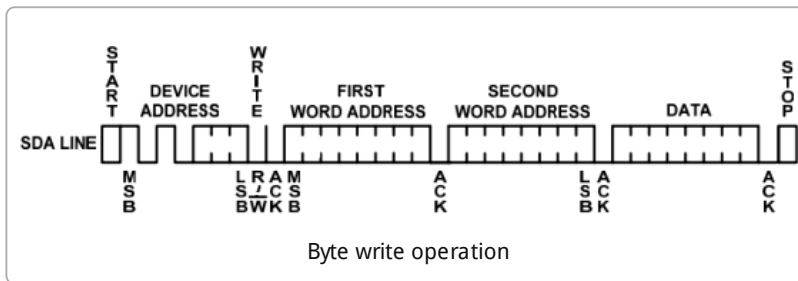
Now let's discuss how these various sequences of operation take place while the PIC18F2550 microcontroller communicates with 24LC512 EEPROM and DS1631 temperature sensor over an I2C bus. The MSSP (Master Synchronous Serial Port) module in PIC18F2550 allows I2C communication through two of its I/O pins: RB0/SDA (21) and RB1/SCL (22). The functioning details of this module is not discussed here because of the use of mikroC compiler that provides library routines for I2C communication.

Serial EEPROM (24LC512)

24LC512 is an I2C compatible serial EEPROM from Microchip Technology with a capacity of 64K x 8 (512 Kbits). The pin diagram of this IC chip is shown in the circuit section. The first four bits of the 7-bit address for this device is set as '1010'. The next three bits are, however, configurable through its A0, A1, and A2 pins. For example, a 24LC512 device with A0 at logic high and A1-A2 pins grounded will have its 7-bit address as '1010001'. This scheme allows a maximum of 8 similar devices addressable on the same I2C bus.

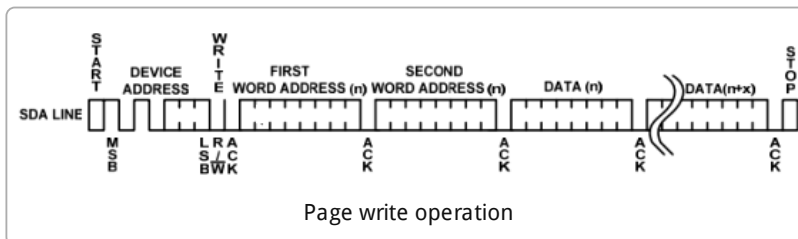
Byte Write operation

For byte write operation, two bytes of addresses are required to select one out of 65536 locations in the EEPROM. Master provides these two address bytes after the control byte has been sent. 24LC512 responds with an acknowledge pulse after receiving each address byte. Master then sends a byte of data to be written in to the memory. Upon receipt of this data, 24LC512 sends an acknowledge pulse. The Master then terminates the data transfer by issuing a Stop condition.



Page Write operation

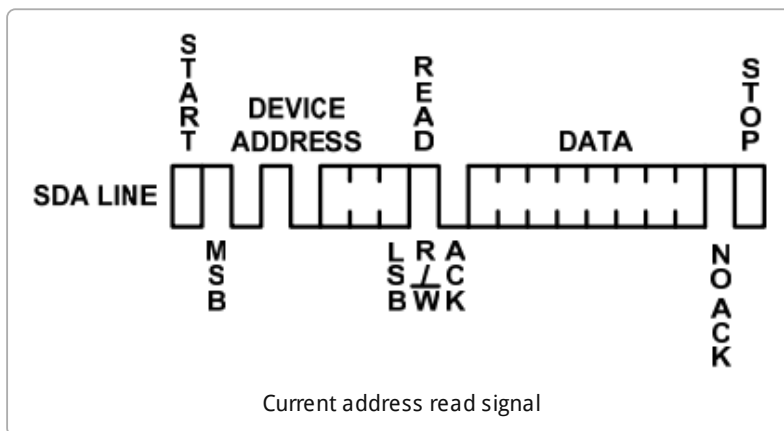
To minimize write cycle time, 24LC512 offers Page write feature, which allows simultaneous writing of up to 128 contiguous bytes. A page write is initiated in the same way as a byte write, but instead of generating a Stop condition, the master transmits up to 127 additional bytes, which are temporarily stored in the on-chip page buffer and will be written into memory after the master has transmitted a Stop condition. Please read the datasheet of 24LC512 to learn more about the Page write operation.



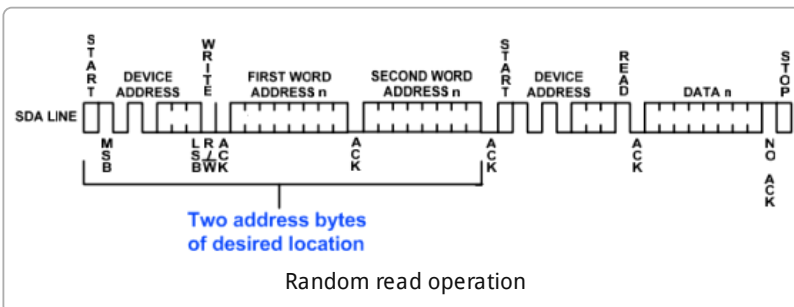
Read operation

Read operations are initiated in the same way as write operations with the exception that the R/W bit of the control byte is set to '1'. 24LC512 allows three basic types of read operations: *current address read*, *random read* and *sequential read*.

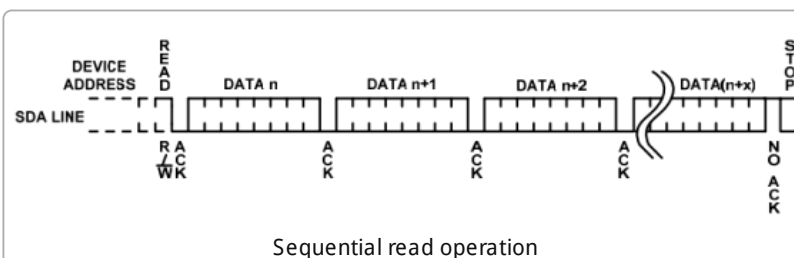
Internally, the EEPROM contains an address counter that maintains the address of the last word accessed, incremented by '1'. Therefore, if the previous read access was to address 'n', the next current address read operation would access data from address $n + 1$. Upon receipt of the control byte with R/W bit set to '1', the EEPROM issues an acknowledge and transmits the current address data byte. The master will not acknowledge the transfer but does generate a following Stop condition and the EEPROM discontinues transmission.



Random read operations allow the master to access any memory location in a random manner. To perform this type of read operation, first the word address must be set. This is done by sending the word address to 24LC512 as part of a write operation (R/W bit set to '0'). After the word address is sent, the master generates a repeated-Start condition following the acknowledge. This terminates the write operation. Then, the master issues the control byte again but with the R/W bit set to a one. The EEPROM will then issue an acknowledge and transmit the 8-bit data word. The master will not acknowledge the transfer but does generate a Stop condition which causes the EEPROM to discontinue transmission.



Sequential reads are initiated by either a current address read or a random address read. After receiving the first byte from 24LC512, the master issues an acknowledge instead of the Stop condition used in a current address or random read. This acknowledge directs 24LC512 to transmit the next sequentially addressed 8-bit word. Following the final byte transmitted to the master, the master will NOT generate an acknowledge, but will generate a Stop condition.



Write Protect (WP)

Pin number 7 of 24LC512 is a hardware Write Protect input. If this pin is tied to Vcc, write operations are inhibited but read operations are not affected. In our experiment, we will ground this pin.

I2C temperature sensor (DS1631)

DS1631 is a digital thermometer manufactured by Dallas Semiconductor (now MAXIM) that provides 9, 10, 11, or 12-bit (user selectable) temperature measurements over a -55 °C to 125 °C. The default resolution at power-up is 12-bit, corresponding to the temperature increment of 0.0625 °C. The communication with DS1631 is achieved through I2C interface and three address pins (A0, A1, and A2) allow up to 8 devices to be multi-dropped on the same 2-wire bus (see the pin diagram in the circuit section). The 7-bit address of each slave is 1 0 0 1 A2 A1 A0, where A0, A1, and A2 are user selectable through the corresponding input pins.

Temperature measurement

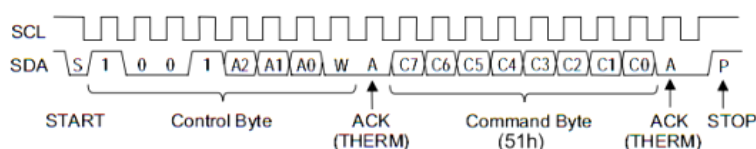
I recommend to read the datasheet of DS1631 for details on its architecture and temperature conversion process. Here, I am describing only the one-shot mode of temperature conversion. Suppose, the device is just powered up, and the resolution of conversion is set to 12-bit. In one-shot mode, the DS1631 sensor starts converting temperature into 12-bit digital word after receiving a command byte, 51h, from the Master. This is known as the Start Convert T command. After the conversion, the digital temperature is stored as a 16-bit two's complement number in to its two-byte temperature register: TH and TL (see below). The Sign bit (S) indicates whether the temperature is positive (S=0) or negative (S=1).

TEMPERATURE, T_H, AND T_L REGISTER FORMAT

	bit 15	bit 14	bit 13	bit 12	bit 11	bit 10	bit 9	bit 8
MS Byte	S	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
LS Byte	2 ⁻¹	2 ⁻²	2 ⁻³	2 ⁻⁴	0	0	0	0

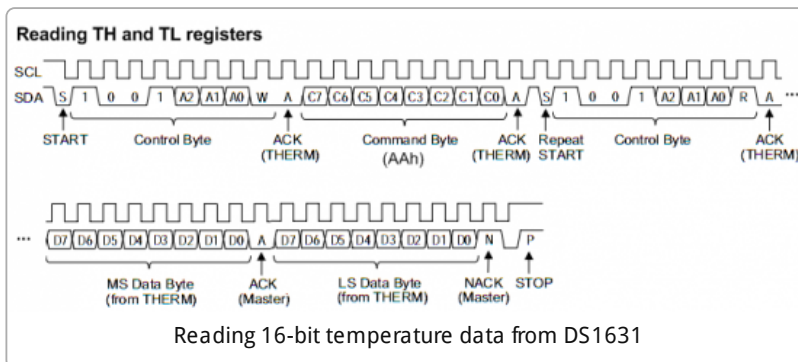
Temperature register inside DS1631

Sending 'Start Convert T' command



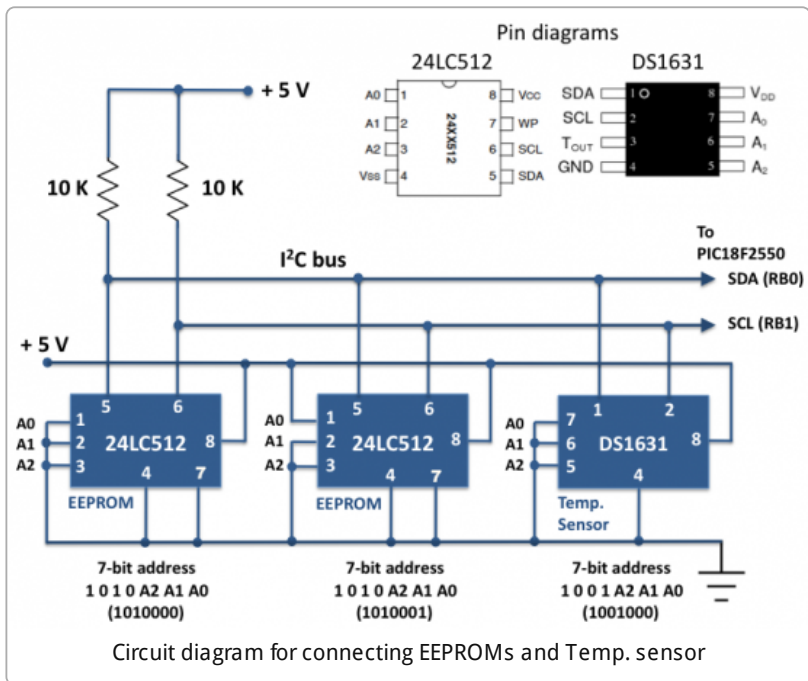
Issuing a Start Convert T command

The master can read the temperature data from DS1631 by sending a Read Temperature (AAh) command. After receiving an ACK in response to the command, the master must generate a repeated Start followed by a control byte with the same slave address as the first control byte. However, this time the R/W bit must be a 1, which tells DS1631 that a “read” is being performed. DS1631 sends an ACK in response to this control byte, and it begins transmitting the requested data on the next clock cycle. For two byte reads (TH and TL registers), the master must respond to the first data byte with an ACK and to the second byte with a NACK followed by a STOP. If only the most significant byte of data is needed, the master can issue a NACK followed by a STOP after reading the first data byte.

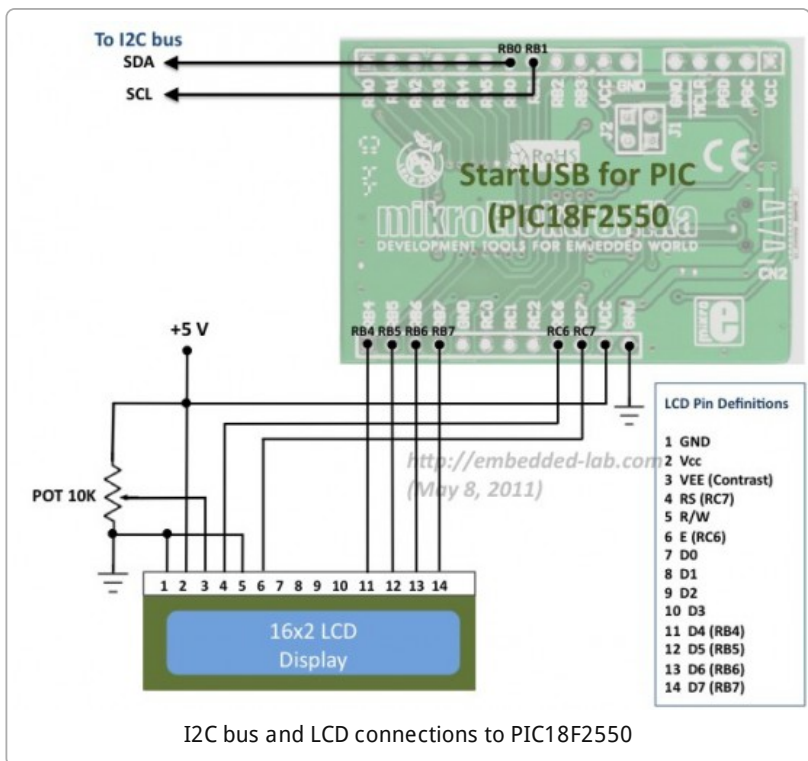


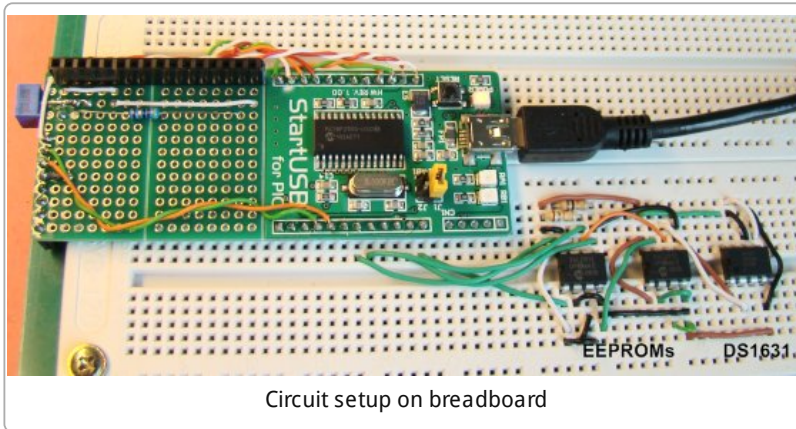
Circuit setup

The circuit diagram for I2C experiment is shown below. Two serial EEPROMs (24LC512) and a DS1631 temperature sensor are connected on a common I2C bus. The address conflict between the two EEPROMs is avoided by connecting one's A0 pin to Gnd and others to Vcc. The 7-bit addresses of all three I2C devices are shown in the diagram. The two 10 K resistors are the pull-up resistors for the I2C bus lines. The SDA and SCL lines go to RB0 and RB1 pins of PIC18F2550, respectively. I am using the [StartUSB for PIC](#) board again for PIC18F2550.



A standard 16×2 character LCD is also used in the circuit to display the I2C Read/Write operations. The microcontroller pins used to drive the LCD are shown below.





Circuit setup on breadboard

Software

We will be writing an application software for testing the I2C communication between the PIC18F2550 microcontroller and the three devices that are multi-dropped on a common I2C bus. The program will first check the presence of all three devices on the bus, and then sends a temperature conversion command to DS1631. The two-byte temperature will be then read and stored in to the two EEPROMs (higher byte in one EEPROM and lower byte in the other, starting at location 0). After five temperature samples are recorded, they will be read sequentially from EEPROM locations, converted in to actual temperature units ($^{\circ}\text{C}$) and displayed on LCD screen.

The mikroC Pro for PIC compiler offers the library routines for handling I2C communication, which are briefly described here.

void I2C1_Init(const unsigned long clock) :- The first thing we need to do is the initialization of the MSSP module for I2C communication. This can be done with *I2C1_Init* function. e.g. *I2C1_Init(100000)* will configure the MSSP module for I2C communication at 100 KHz.

unsigned short I2C1_Start(void) :- Determines if I²C bus is free and issues Start signal. It returns zero if there is no error.

void I2C1_Repeated_Start(void) :- Issues a repeated-start condition.

unsigned short I2C1_Is_Idle(void) :- Checks if I2C bus is free. If it is free, it returns 1, otherwise 0.

unsigned short I2C1_Rd(unsigned short ack) :- Reads one byte from the slave, and sends *NACK* signal if parameter *ack* is 0, otherwise it sends *acknowledge*. For example, if the microcontroller needs to read two-byte temperature readings from DS1631, it should use *I2C1_Rd(1)* to read the first byte and *I2C1_Rd(0)* for the second byte.

unsigned short I2C1_Wr(unsigned short data) :- Sends data byte via I2C bus. It returns zero if there is no error.

void I2C1_Stop(void) :- Issues a Stop condition.

In our test program, PIC18F2550 will first check the availability of the three devices on the bus. It is done by sending a device address for each and if it is acknowledged, that means the device is present. A mikroC routine to check the presence of a device on the I2C bus is given below.

```
void check_device(unsigned short dev_address){
    I2C1_Start();
    if (I2C1_Wr(dev_address)){
        Lcd_Out(2,1,"Device not found");
    }
    else Lcd_Out(2,1,"Device is OK");
    I2C1_Stop();
}
```

The following program allows PIC18F2550 to send a Start Convert T command to DS1631, then wait for 750 ms until the 12-bit temperature conversion is done, and finally receive the two-byte temperature reading.

```
// Read temperature
I2C1_Start();
I2C1_Wr(DS1631);           // Send device address
I2C1_Wr(0x51);             // Start Convert Temp command
I2C1_Stop();
Delay_ms(750);
I2C1_Start();
I2C1_Wr(DS1631);           // DS1631 Address again
I2C1_Wr(0xAA);             // Read Temperature command
I2C1_Repeated_Start();
I2C1_Wr(DS1631+1);         // Address with Read
MS_Byte = I2C1_Rd(1);      // Read MSB and send ACK
LS_Byte = I2C1_Rd(0);      // Read LSB and send NAK
I2C1_Stop();
```

You can see while reading the last byte, PIC18F2550 does not need to send an acknowledge pulse.

The following two routines will perform byte read and write operations with the two EEPROMs.

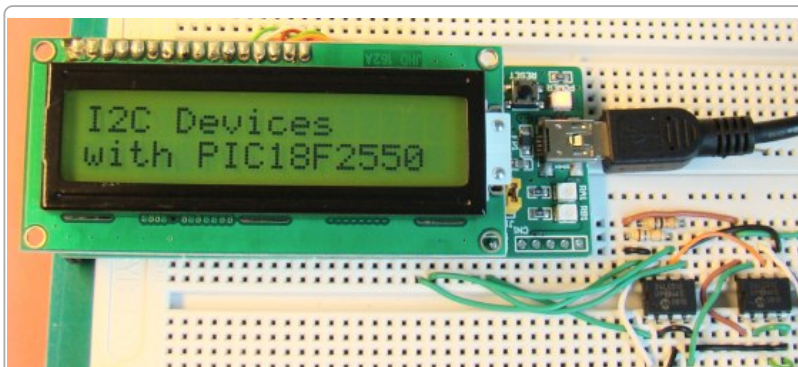
```
//----- Reads data from 24LC512 EEPROM - single location
unsigned short EEPROM_ReadByte(unsigned short EEPROM_Select, unsigned short rAddress) {
    unsigned short rAddrH, rAddrL, result;
    rAddrH = 0;
    rAddrL = rAddress;
    I2C1_Start();           // issue I2C1 start signal
    I2C1_Wr(EEPROM_Select); // send byte via I2C1 (device address + W)
    I2C1_Wr(rAddrH);        // send Higher address byte
    I2C1_Wr(rAddrL);        // send Lower address byte
    I2C1_Repeated_Start();  // issue I2C1 signal repeated start
    I2C1_Wr(EEPROM_Select+1); // send byte (device address + R)
```

```
result = I2C1_Rd(0u);    // Read the data (NO ack  
nowledge)  
while (!I2C1_Is_Idle())  
asm nop;                // Wait for the read cycle  
to finish  
I2C1_Stop();  
return result;  
}  
  
//----- Writes data to 24LC512 EEPROM - s  
ingle location  
void EEPROM_WriteByte(unsigned DeviceSelect, unsign  
ed short wAddress, unsigned short wData) {  
    unsigned short wAddrH, wAddrL;  
    wAddrH = 0;  
    wAddrL = wAddress;  
    I2C1_Start();        // issue I2C1 start signal  
    I2C1_Wr(DeviceSelect); // send control byte  
    I2C1_Wr(wAddrH);      // send higher address byte  
    I2C1_Wr(wAddrL);      // send lower address byte  
    I2C1_Wr(wData);       // send data to be written  
    I2C1_Stop();  
}
```

[Download mikroC source and hex files](#)

Output

The following pictures show the sequence of operations performed while executing the I2C test software.



Start up message



Testing EEPROM1